

Problem 1 S-B 树

100: 直接模拟。

不难发现 S-B 树有排序二叉树的性质:

每个节点的左子树的所有节点都小于该节点, 右子树的所有节点都大于该节点。

排序二叉树中搜索 m/n , 同时输出路径。

时间复杂度等于 LR 序列的长度。

顺带一提, 可以证明, LR 序列的长度不超过 $\max\{n, m\}$, 且和辗转相除法的兄弟, 更相减损术的迭代次数相同。参见 `a_std.cpp`

Problem 2 匹配括号串

50: $O(n!)$ 暴力做法。

100: 直接 DP。

设 $dp[i][j]$ 表示长度为 i , 深度不超过 j 的匹配括号串个数。

我们在 $D(e)$ 的定义中已经提示:

非空串 s 可以唯一写成 $s = (l)r$,

枚举 l 的长度 k , $0 \leq k \leq i - 2$,

$$dp[i][j] = \sum_{k=0}^{i-2} dp[k][j-1] \cdot dp[i-2-k][j]$$

先一次计算 $dp[i][j]$, 然后 $O(1)$ 查询答案 $ans = dp[n][d] - dp[n][d-1]$ 。时间复杂度 $O(n^2 + t)$, 参见 `c_std.cpp`

Problem 3 成员分配

特殊做法: 网络流。本题可以建模成最大流问题。具体建图参见 `c_dinic.cpp`。

正常做法: 贪心。

贪心策略: 把队员优先安排到剩余容量大的桌子上。

证明: 见附录 A。

直接贪心, 桌子容量用优先队列维护, 时间复杂度是 $O(\sum \max\{a_i, m\} \log m) = O(nm \log m)$ 。参见 `c_std.cpp`

Problem 4 智慧城市

本题来自李煜东《算法竞赛进阶指南》例题《天才 ACM》。

给定 m , 序列 $[a_1, \dots, a_n]$ 升序排序, 则波动值为 $\sum_{i=1}^{\min\{m, n/2\}} (a_i - a_{n-i+1})^2$ (请读者自行完成证明)。

固定区间左端点, 波动值 checksum 相对右端点单调递增。

维护 $[l, r]$, 其中 l 是当前段的开始, $\text{checksum}([l, r]) \leq k$ 。

计算出最大的 r , 开启新段 $l, r \leftarrow r + 1$ 。

如果递增 r , 用插入排序维护。递增 n 次, 时间复杂度 $O(n^2)$ 。

如果 m 较小, 只要分别维护 $[l, r]$ 中最大和最小的 m 个元素即可。时间复杂度 $O(nm)$ 。

还有 $k = 0, 1$ 的特殊情况, 暴力做法能获得 80% 分数。

为了加速查找 r 的过程, 需要用倍增。倍增的策略有多种, 下面介绍一种便于理解的。

设 r 的初始值为 l , $d = 0$ 。递增 d , 直到 $\text{checksum}([l, r + 2^d]) > k$ 。

取最大的 d , 即 $r \leftarrow r + 2^{d-1}$ 。递减 d , 根据 $\text{checksum}([l, r + 2^d])$ 的值, 决定是否 $r \leftarrow r + 2^d$

时间复杂度分析: 设 $[1, n]$ 需要分成 $N_1, N_2, \dots, N_l$ 。在 N_i 中, 倍增分为扩张期和收缩期。

扩张期的 d 增大 $\log N_i$ 次, 计算 checksum : $O(N_i \log N_i)$ 。收缩期对称。

总时间复杂度 $\sum_{i=1}^l O(N_i \log^2 N_i) = O(n \log^2 n)$ 。

可以优化排序, $[l, r]$ 在上一次检查中已经是升序的, 检查 $[l, r + 2^d]$ 时, 只需要排序 $[r + 1, r + 2^d]$, 再用归并排序中的合并。

排序 $d = 0.. \log n$ 的长度为 2^d 的序列的时间复杂度为 $\sum_{i=0}^{\log n} O(2^i \log 2^i) = O(n \log n)$ 。

总体时间复杂度 $O(n \log n)$ 。参见 `d_std.cpp` 倍增细节和题解有些许不同。

附录 A p3 贪心策略正确性证明

定义: $a \rightarrow b :=$ 能把人数为 a 的若干团队安排到容量为 b 的若干桌子上。

即 $\forall a = [a_1, a_2, \dots, a_n], b = [b_1, b_2, \dots, b_m], a \rightarrow b$

$$\iff \exists f_1, f_2, \dots, f_n, \forall i, \forall k \in [1, a_i], k_1 \neq k_2 \implies f_i(k_1) \neq f_i(k_2) \wedge \forall j, \sum_{i, k \text{ s.t. } f_i(k)=j} 1 \leq b_j$$

定义 $\text{sort}([b_1, b_2, \dots, b_m]) = [b_{p_1}, b_{p_2}, \dots, b_{p_m}]$, 其中 $b_{p_1} \leq b_{p_2} \leq \dots \leq b_{p_m}$, p 是 1 到 m 的一个排列。

显然 $a \rightarrow b$ 与 b 的排列顺序无关。

贪心判定定理: $[a_1, \dots, a_n] \rightarrow b \iff [a_2, \dots, a_n] \rightarrow \text{sort}(b) - \underbrace{[0, \dots, 0]_{m-a_1}}_{a_1} \underbrace{[1, \dots, 1]_{a_1}}_{a_1}$ 。

定义 $b \vdash b' := b$ 和 b' 的总和相同, 最多有 2 对元素 b_i, b_j 和 b'_i, b'_j 不同, 满足 $b_i > b_j, b'_i = b_i - 1, b'_j = b_j + 1$ 。

若存在 $b_0, \dots, b_k$ 使得 $b = b_0 \vdash b_1 \vdash \dots \vdash b_k = b'$, 则称 $b \vdash b'$ 。

判定定理的必要性 ($\Leftarrow$) 显然。

充分性 ($\Rightarrow$),

把 $a \rightarrow b$ 的定义中给出的 f_1 写成 $\delta_1(i) = \begin{cases} 1 & \text{if } i \in \text{Im} f_1 \\ 0 & \text{otherwise} \end{cases}$

可得 $[a_2, \dots, a_n] \rightarrow b - \delta_1$ 。易证 $b - \delta_1 \vdash \text{sort}(b) - \underbrace{[0, \dots, 0]_{m-a_1}}_{a_1} \underbrace{[1, \dots, 1]_{a_1}}_{a_1}$ 。

只要证明引理: 若 $a \rightarrow b$, 则 $\forall b' \text{ s.t. } b \vdash b', a \rightarrow b'$ 。

不失一般性, 设 $b = [b_1, \dots, b_i, \dots, b_j, \dots, b_m], b' = [b_1, \dots, b_i - 1, \dots, b_j + 1, \dots, b_m], b_i > b_j$ 。

取 $a \rightarrow b$ 定义中的 f 。若桌位 i 的人数 $\sum_{l, k \text{ s.t. } f_l(k)=i} 1 < b_i$, 则 $a \rightarrow b'$ 。

否则, $\sum_{l, k \text{ s.t. } f_l(k)=i} 1 = b_i$, 则 $\sum_{l, i \in \text{Im} f_l} 1 > b_j \geq \sum_{l, j \in \text{Im} f_l} 1$ 。

根据鸽巢原理, 存在某个团队 l 的某个人 $k, f_l(k) = i$, 且 $j \notin \text{Im} f_l$ 。

则 $f' = f - f_l + f_l|_{k \rightsquigarrow j}$ 证明了 $a \rightarrow b'$ 。

至此, 我们证明了判定定理。□

贪心策略就是不断应用判定定理, 直到 a 为空或者 b 不够分配。

形象地说, 引理告诉我们, 桌子的容量分布越均匀, 人员分配越容易。

存在一个最“均匀”分配方向, 即满足最优子结构性。

附录 B 文件一览表

文件名	说明
2025PS2_Final.*	题目
README.md	本文档